

2026-09-28 更新：gate v3 的能力剖析——它到底在判断什么

一、这次要回答的问题

生产检索里用的 gate (gate v3, `gate/gate_v3/artifacts_pooled_v3`) 一直只有一个整体评价：「加上它，端到端指标涨了多少」。但它逐跳输出的那个异常分本身有什么性质，从来没有系统测过，特别是：

- 一条 trace 里非 gold 的证据越多，分数怎么变？
- 一跳选错之后，后面那些本身选对了的跳，分数会不会被带高？
- gate v3 的特征是 `concat(PCA(h_after), PCA( $\Delta$ ))` 两半拼起来的，判别力来自哪一半？

这三个问题在生产数据上答不了：`traces/construct_balanced_traces.py` 的构造协议是「单跳注入」，`wrong_hops` 长度 100% 为 1，没有多错场景；而且 `hidden_states/extract_hidden_states.py` 里 `n_take = wrong_hop + 1`，出错那一跳之后就不再抽隐状态，所以现有的 9.2 万个 `.npz` 里没有任何「错误发生之后」的隐状态。

二、实验设置

数据 (验证集全量)：对 MuSiQue dev 的每道题 (K 跳) 构造 `n_wrong = 0, 1, ..., K` 一整套变体，被污染的那几跳把 gold 证据换成 cosine top-1 的非 gold 段落 (和生产构造负样本用的是同一套 `CosinePoolRanker` 逻辑)。K=3、K=4 的题把错误位置的组合枚举完整。

	题数	trace 数	n_wrong 分布
K=2	1,252	3,756	0/1/2 各 1,252
K=3	760	6,080	0:760、1:2,280、2:2,280、3:760
K=4	405	6,480	0:405、1:1,620、2:2,430、3:1,620、4:405
合计	2,417	16,316	0:2,417、1:5,152、2:5,962、3:2,380、4:405

打分方式跟生产逐字一致：转移 j 用 `meta["per_j_layers"][j]` 指定的层 (j=0/1 取第 15 层，j=2/3 取第 23 层)，特征是 `concat(PCA(h_{j+1}), PCA(h_{j+1} - h_j))`，输出逻辑回归的 P(异常)——也就是生产重排时从 cosine 分里减掉的那一项。gate 只有 j=0..3 四个模型，所以每条 trace 计分的是前 4 跳。最终 51,672 个跳全部打分成功，没有缺层、没有对不上的行。

gate v3 训练用的是 MuSiQue train 的 trace，这里评的是 dev，题目完全不重叠。阈值沿用训练时按 target FPR=0.15 定出来的值 (j0=0.267、j1=0.380、j2=0.461、j3=0.345)。

三、结果

3.1 分数 vs 前缀里的错误数

格式为 `均值±标准差 (跳数) / 超阈值比例`。

汇总

前缀中错误数	当前跳是错的	当前跳是对的
0	—	0.169±0.226 (14,537) / 0.17
1	0.766±0.302 (13,899) / 0.85	0.407±0.334 (8,109) / 0.43
2	0.708±0.304 (8,747) / 0.79	0.461±0.334 (2,785) / 0.50
3	0.703±0.299 (2,785) / 0.80	0.556±0.351 (405) / 0.66
4	0.766±0.274 (405) / 0.89	—

按题目跳数拆开

K	前缀中错误数	当前跳是错的	当前跳是对的
2	0	—	0.172 (3,142) / 0.19
	1	0.779 (2,504) / 0.86	0.506 (614) / 0.57
	2	0.758 (1,252) / 0.85	—
3	0	—	0.168 (5,320) / 0.17
	1	0.777 (5,320) / 0.87	0.408 (3,040) / 0.41
	2	0.689 (3,040) / 0.77	0.447 (760) / 0.45
	3	0.683 (760) / 0.76	—
4	0	—	0.168 (6,075) / 0.17
	1	0.750 (6,075) / 0.83	0.392 (4,455) / 0.42
	2	0.708 (4,455) / 0.79	0.467 (2,025) / 0.51
	3	0.710 (2,025) / 0.81	0.556 (405) / 0.66
	4	0.766 (405) / 0.89	—

两个读数：

- **误报率**和设计值吻合：全对前缀下的正确跳，17% 超阈值，而训练时设定的 target FPR 是 0.15。三个 K 上都是 0.17-0.19，说明这个阈值迁移到没见过的 dev 数据上仍然成立。
- **检出率**：被污染的那一跳有 79%-89% 超阈值，平均分 0.70-0.78。
- **正确跳的分数随前缀错误数单调上升**：0.169 → 0.407 → 0.461 → 0.556，超阈值比例 0.17 → 0.43 → 0.50 → 0.66。三个 K 的趋势一致。
- **检测强度不随错误增多而变强，反而略降**：「当前跳是错的」这一列是 0.766 → 0.708 → 0.703，前缀已经脏了以后，gate 对「这一跳又错了」的判断没有更敏锐。

3.2 相对第一个错误跳的位置

只看 $n_wrong \geq 1$ 的 trace。「其中本身是对的」一列把该位置上证据本身正确的跳单独拿出来。

位置	全部	其中本身是对的
错误前第 3 跳	0.125 (405)	0.125 (405)
错误前第 2 跳	0.135 (1,975)	0.135 (1,975)
错误前第 1 跳	0.164 (5,753)	0.164 (5,753)
错误发生的那一跳	0.766 (13,899)	—（构造上必然为空）
错误后第 1 跳	0.591 (12,096)	0.457 (5,729)
错误后第 2 跳	0.542 (7,900)	0.395 (3,950)
错误后第 3 跳	0.569 (3,240)	0.389 (1,620)
全 gold trace（对照）	0.186 (6,404)	—

这是全文最关键的一组对照：**错误之前的跳没有被抬高**（0.125-0.164，和全对基线 0.169 持平），**错误之后本身选对的跳被抬高到 2.3-2.7 倍**（0.389-0.457）。同一条 trace 内部前后对比，排除了「这些题本身更难」的解释。而且抬高之后基本不衰减：错误后第 3 跳仍是 0.389。

按 K 拆开趋势一致，例如错误后第 1 跳、本身选对的：K=2 是 0.506、K=3 是 0.453、K=4 是 0.450。

3.3 控制跳的位置

上面的抬高会不会只是因为「前缀更长、跳数更靠后」？把跳的位置固定住再看：

K	跳	本跳是错的	本跳对、前缀全对	本跳对、前缀有错
2	1	0.784 (1,866)	0.172 (1,890)	—
2	2	0.759 (1,890)	0.171 (1,252)	0.506 (614)
3	1	0.823 (3,040)	0.123 (3,040)	—
3	2	0.718 (3,040)	0.233 (1,520)	0.506 (1,520)
3	3	0.678 (3,040)	0.217 (760)	0.356 (2,280)
4	1	0.785 (3,240)	0.125 (3,240)	—
4	2	0.720 (3,240)	0.175 (1,620)	0.464 (1,620)
4	3	0.671 (3,240)	0.303 (810)	0.419 (2,430)
4	4	0.744 (3,240)	0.209 (405)	0.405 (2,835)

同一个位置上，前缀有错时的正确跳仍然显著高于前缀全对时（如 K=4 第 4 跳：0.405 vs 0.209），所以 3.2 的结论不是跳数位置带来的假象。

这张表另外说明一件事：**越靠后的跳，信噪比越差**。检出强度从第 1 跳的 0.78-0.82 降到第 3 跳的 0.67-0.68，而全对前缀下的误报从 0.12-0.13 升到 0.21-0.30。j=2、j=3 两个模型的训练样本本来就比 j=0、j=1 少得多。

3.4 trace 级别

K	n_wrong	trace 数	该 trace 各跳分数的均值	各跳分数的最大值
2	0	1,252	0.171	0.275
2	1	1,252	0.557	0.816
2	2	1,252	0.770	0.910
3	0	760	0.191	0.379
3	1	2,280	0.428	0.802
3	2	2,280	0.610	0.903
3	3	760	0.739	0.940
4	0	405	0.203	0.495
4	1	1,620	0.383	0.817
4	2	2,430	0.533	0.909
4	3	1,620	0.651	0.944
4	4	405	0.733	0.959

Spearman(n_wrong, 分数均值) = 0.606, Spearman(n_wrong, 分数最大值) = 0.508。

均值随 n_wrong 单调上升有一部分是机械的（错的跳多了，高分的跳自然多），真正非平凡的是 3.1 里 正确跳那一列。最大值这一列另有一个值得注意的现象：**全 gold trace 的最大分随 K 上升**（K=2 是 0.275，K=3 是 0.379，K=4 是 0.495），而只错一跳的 trace 大约在 0.80-0.82。也就是说 用「整条路径上的最高分」判断路径是否被污染，2 跳题上区分度很大，4 跳题上明显变窄——跳数越多，撞上一次误报的概率越大。

3.5 fusion 的两半，判别力来自哪一半

gate v3 的 logit 可以精确拆开： $\text{logit} = \text{截距} + w_h \cdot \text{PCA}(h_after) + w_\Delta \cdot \text{PCA}(\Delta)$ 。

类别	跳数	分数	logit(h)	logit(Δ)	截距
正确跳，前缀全对	14,537	0.169	−1.62	−0.31	−0.57
正确跳，前缀有 1 错	8,109	0.407	−0.08	−0.45	0.04
正确跳，前缀有 2+ 错	3,190	0.474	−0.14	−0.31	0.36
错误跳，前缀无其他错	13,899	0.766	+3.20	−0.04	−0.58
错误跳，前缀有其他错	11,937	0.709	+1.88	−0.26	0.11

这张表把 4 个 j 的模型混在一起平均了（截距从 −0.58 到 +0.36 就是混合比例在变），所以按 j 分开 再算一次对错分离度（Cohen's d，抽样 1/4 的 trace）：

j	转移	h 那一半	Δ 那一半
0	Q→E1	2.20	0.18

j	转移	h 那一半	Δ 那一半
1	E1→E2	1.70	0.47
2	E2→E3	1.43	0.07
3	E3→E4	1.83	0.39

四个模型一致：判别力几乎全部来自 **h 那一半**， **Δ 那一半**贡献很小。而 **h 那一半**在三类上的取值（以 $j=1$ 为例：前缀全对的正确跳 -1.41 、前缀有错但本跳正确 $+0.82$ 、本跳错误 $+1.97$ ）显示，「前缀脏了但这一跳是对的」落在两者中间，而不是贴近「正确」那一侧。

准确的表述是：在已经有 **h** 的情况下， **Δ** 没有提供额外的判别力，而不是 **Δ** 本身没有信息量——两半特征并不独立（ $\Delta_j = h_j - h_{j-1}$ ，与 h_j 共用一项）。

仓库里已有的三个 gate 变体在同配置（ $\lambda=0.5$ ，dev 全量）下的端到端结果，从另一个角度印证了这件事：

数据集	只用 Δ (v2)	只用 h	h+ Δ (v3, 现用)
MuSiQue recall@1 / EM	0.6846 / 0.4150	0.6977 / 0.4245	0.7023 / 0.4307
2WikiMultihopQA	0.9119 / 0.4912	0.9244 / 0.4967	0.9289 / 0.4990
HotpotQA	0.6557 / 0.5175	0.6573 / 0.5183	0.6596 / 0.5217

只用 **h** 已经接近 **v3**（差 0.5pt 以内），只用 **Δ** 明显更差。

四、对这个 gate 能力的判断

1. 它是一个「前缀被污染了没有」的检测器，不是「哪一跳错了」的定位器。

这是把 3.2、3.3、3.5 串起来之后的核心结论。特征的主力是 **h_{after}** ，即整条前缀的绝对状态；前缀里只要有一处污染，它就报高分，无论污染发生在哪一跳、当前这一跳选得对不对。三个原本看起来独立的现象因此有了同一个解释：

- 错误之后的正确跳被抬高——不是「错误的痕迹残留在隐状态里」这种偶然现象，而是这个特征必然的行为；
- 错误越多、检出强度不升反降——它本来就不在定位「哪一跳错」，前缀一旦脏了，再脏一点没有新信息；
- 单跳注入的训练数据下它也能学好——因为它学的其实是「前缀正常 vs 前缀不正常」这个更容易的区分。

2. 作为生产用的重排信号，它是有效的。 检出 85%、误报 17%，且阈值从 train 迁移到 dev 后误报率仍守在设计值附近。这解释了它在端到端上的收益（0831 报告 1.5 节， $\lambda=0.60$ ，相比不用 gate）：chain@1 在三个数据集上提升 +17.8 (MuSiQue)、+26.4 (2Wiki)、+22.5pt (HotpotQA)，EM 提升 +6.3、+14.8、+6.0pt。提升集中在 chain@1，正符合它的性质——它擅长把「整条链里某一跳的错」纠回来。

3. 但它同时会放大错误。 一旦某跳选错，后面本身正确的候选有 43%–50% 被判成异常（全对基线是 17%）。生产是贪心逐跳推进，这意味着错误之后 gate 会持续压低正确候选。而且由于它不能定位错误位置，也无法用来支撑「回退到出错那一跳重选」这类策略。**这个副作用和它的主要能力是同一件事的两面：**对整条路径的好坏敏感，就必然对「路径里已有的错」过度反应。

4. 换个用法反而更契合它的性质。 它天然提供的是路径级信号： n_wrong 越大分数越高（Spearman 0.606），且错误在后续跳里不衰减。也就是说，比较多条候选路径整体好坏这件事，它本身 就在做。需要注意的是 3.4 里的那个现象——全 gold trace 的最高分随 K 上升（K=4 时 0.495），所以 用最高分做路径判别时，跳数越多区分度越窄。

5. 后段跳是薄弱环节。 检出从第 1 跳的 0.78-0.82 降到第 3 跳的 0.67-0.68，误报从 0.12-0.13 升到 0.21-0.30。四跳题的后两跳既最难检出、又最容易误报，而这正是错误累积最严重的位置。

一个口径上的提醒：本次被替换的错误证据是 cosine top-1 的非 gold 段落，与 gate 训练时构造负样本 的方式相同。生产中的错误证据来自真实检索排序，分布不完全一致，所以 79%-89% 这个检出率应当视为 乐观估计。

五、文档需要同步的一处

CLAUDE.md 里现在的描述是「The gate operates on Δ features: $\Delta_j = h_j - h_{\{j-1\}}$ 」。这是 v2 时代的写法：v3 的特征是 `concat(PCA(h_after), PCA( $\Delta$ ))`，而且按 3.5 的拆解，实际起作用的是 h 那一半。